

MANUAL TÉCNICO

Sistema de Gestión de Aeropuerto

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Estructuras de Datos - Sección A

Carné: 202303204

1. Descripción del Sistema

El Sistema de Gestión de Aeropuerto es una aplicación de consola desarrollada en C++ que permite gestionar vuelos, aviones, pasajeros y equipajes mediante estructuras de datos implementadas manualmente. El sistema simula operaciones de un aeropuerto incluyendo registro de pasajeros, mantenimiento de aviones y facturación de equipajes.

Características principales:

- Gestión de aviones disponibles y en mantenimiento mediante listas circulares dobles
 - Registro de pasajeros con cola FIFO y lista ordenada
 - Manejo de equipajes mediante pila LIFO
 - Generación de reportes visuales con Graphviz (.dot y .png)
 - Carga de datos desde archivos JSON
 - Procesamiento de comandos desde archivo de texto
-

2. Requisitos del Sistema

2.1 Sistema Operativo

- Windows 10/11 (64-bit)
- Linux (distribuciones basadas en Debian/Ubuntu o similares)
- macOS (10.14 o superior)

2.2 Software Requerido

Compilador C++

- MinGW-w64 (Windows) con soporte para C++11 o superior
- GCC/G++ versión 7.0 o superior (Linux/macOS)

Graphviz

- Graphviz 2.38 o superior
- Debe estar instalado y accesible desde la línea de comandos
- Comando `dot` debe estar en el PATH del sistema

Instalación en Windows:

```
# Descargar desde: https://graphviz.org/download/  
# Agregar al PATH: C:\Program Files\Graphviz\bin
```

Instalación en Linux (Ubuntu/Debian):

```
sudo apt-get update  
sudo apt-get install graphviz
```

Instalación en macOS:

```
brew install graphviz
```

Dependencias de C++

- Librería `nlohmann/json.hpp` (incluida en `src/json/json.hpp`)
- Soporte UTF-8 para caracteres especiales en español

3. Estructura del Proyecto

```
202303204_EDD_Practica/  
|  
├─ data/  
|   ├─ aviones.json          # Datos iniciales de aviones  
|   ├─ pasajeros.json       # Datos iniciales de pasajeros  
|   └─ Cambios.txt           # Comandos de movimientos  
|  
├─ docs/  
|   ├─ Enunciado_Practica_EDD.pdf  
|   └─ MANUAL_TECNICO.md     # Este documento  
|
```

```

├─ reportes-generados/
│   └─ (archivos .dot y .png generados por el sistema)
│
└─ src/
    ├─ main.cpp          # Punto de entrada del programa
    ├─ Menu.cpp          # Implementación del menú
    ├─ Menu.h            # Interfaz del menú
    ├─ compile.bat       # Script de compilación (Windows)
    │
    ├─ controllers/
    │   ├─ AvionController.h
    │   ├─ PasajeroController.h
    │   ├─ EquipajeController.h
    │   └─ MovimientosController.h
    │
    ├─ estructuras/
    │   ├─ Nodo.h
    │   ├─ ListaCircularDoble.h
    │   ├─ ListaDoble.h
    │   ├─ Cola.h
    │   └─ Pila.h
    │
    ├─ models/
    │   ├─ Avion.h
    │   ├─ Pasajero.h
    │   └─ Equipaje.h
    │
    └─ json/
        ├─ json.hpp      # Librería nlohmann/json
        └─ JsonParser.h  # Parser personalizado

```

4. Estructuras de Datos Implementadas

4.1 Lista Circular Doble (`ListaCircularDoble.h`)

Uso: Almacenar aviones disponibles y en mantenimiento.

Características:

- Nodos con punteros `siguiente` y `anterior`

- El último nodo apunta al primero (circular)
- Métodos: `insertar()`, `buscar()`, `eliminar()`, `extraer()`, `generarReporte()`

Complejidad temporal:

- Inserción: $O(1)$
- Búsqueda: $O(n)$
- Eliminación: $O(n)$

```

1  #ifndef LISTACIRCULARDOBLE_H
2
3  4  #include <iostream>
5  #include <fstream>
6  #include <cstdlib>
7  #include <string>
8  #include "Nodo.h"
9
10 using namespace std;
11
12 template <typename T>
13 class ListaCircularDoble
14 {
15 private:
16     Nodo<T> *primero;
17     Nodo<T> *ultimo;
18
19 public:
20     ListaCircularDoble()
21     {
22         primero = nullptr;
23         ultimo = nullptr;
24     }
25
26     bool estaVacia()
27     {
28         return (primero == nullptr && ultimo == nullptr);
29     }
30
31     void insertar(T *dato) ...
32
33     T *buscar(string identificador) ...
34
35     bool eliminar(string identificador) ...
36
37     // Mover un elemento a otra lista
38     T *extraer(string identificador) You, 12 hours ago • estructuras fix
39     {
40         if (estaVacia()) ...
41
42         Nodo<T> *actual = primero; // iniciamos desde el primer
43
44         do ...
45         } while (actual != primero);
46
47         return nullptr;
48     }
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83

```

4.2 Cola (Cola.h)

Uso: Cola de registro de pasajeros (FIFO - First In First Out).

Características:

- Punteros `frente` y `final`

- Métodos: `encolar()`, `desencolar()`, `estaVacia()`, `generarReporte()`

Complejidad temporal:

- Encolar: $O(1)$
- Desencolar: $O(1)$

```

1  #ifndef COLA_H
2  #define COLA_H
3
4  #include <iostream>
5  #include <fstream> // para manejo de archivos
6  #include <cstdlib> // para usar el system
7  #include <string>
8  #include "Nodo.h"
9
10 using namespace std;
11
12 template <typename T>
13
14 /*
15  Cola FIFO, el primer elemento en entrar es el primero en salir.
16 */
17
18 class Cola
19 {
20 private:
21     Nodo<T> *primero; // Frente de la cola
22     Nodo<T> *ultimo; // Final de la cola
23 public:
24     cola()
25     {
26         primero = nullptr;
27         ultimo = nullptr;
28     }
29
30     bool estaVacia()
31     {
32         return (primero == nullptr && ultimo == nullptr);
33     }
34
35     // ver el inicio de la cola
36     T *frente()
37     {
38         if (estaVacia())
39         {
40             return nullptr;
41         }
42         return primero->getDato();
43     }
44
45     // saber la cantidad de elementos en la cola
46     int contar()
47     {

```

4.3 Lista Doble Enlazada (`ListaDoble.h`)

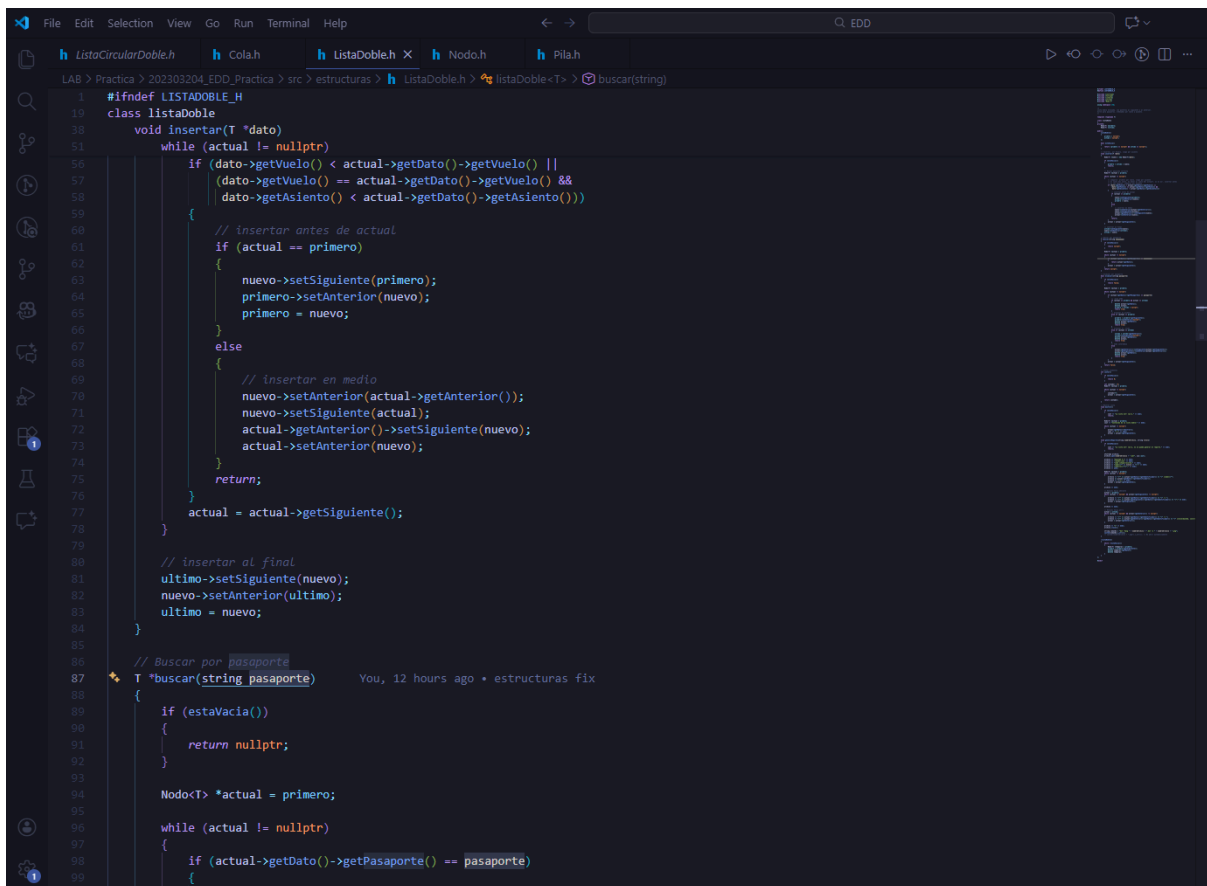
Uso: Almacenar pasajeros registrados ordenados por vuelo y asiento.

Características:

- Nodos con punteros `siguiente` y `anterior`
- Inserción ordenada automática por:
 - Número de vuelo (primario)
 - Número de asiento (secundario)
- Métodos: `insertar()`, `buscar()`, `eliminar()`, `generarReporte()`

Complejidad temporal:

- Inserción ordenada: $O(n)$
- Búsqueda: $O(n)$



```
1 #ifndef LISTADOBLE_H
2 #define LISTADOBLE_H
3
4 class listaDoble
5 {
6     void insertar(T *dato)
7     {
8         while (actual != nullptr)
9         {
10             if (dato->getVuelo() < actual->getDato()->getVuelo() ||
11                 (dato->getVuelo() == actual->getDato()->getVuelo() &&
12                  dato->getAsiento() < actual->getDato()->getAsiento()))
13             {
14                 // insertar antes de actual
15                 if (actual == primero)
16                 {
17                     nuevo->setSiguiente(primero);
18                     primero->setAnterior(nuevo);
19                     primero = nuevo;
20                 }
21                 else
22                 {
23                     // insertar en medio
24                     nuevo->setAnterior(actual->getAnterior());
25                     nuevo->setSiguiente(actual);
26                     actual->getAnterior()->setSiguiente(nuevo);
27                     actual->setAnterior(nuevo);
28                 }
29                 return;
30             }
31             actual = actual->getSiguiente();
32         }
33
34         // insertar al final
35         ultimo->setSiguiente(nuevo);
36         nuevo->setAnterior(ultimo);
37         ultimo = nuevo;
38     }
39
40     // Buscar por pasaporte
41     T *buscar(string pasaporte)
42     {
43         if (estaVacia())
44         {
45             return nullptr;
46         }
47
48         Nodo<T> *actual = primero;
49
50         while (actual != nullptr)
51         {
52             if (actual->getDato()->getPasaporte() == pasaporte)
53             {
54                 return actual;
55             }
56             actual = actual->getSiguiente();
57         }
58         return nullptr;
59     }
60
61     // ... (other methods)
62 }
63
64 #endif
```

4.4 Pila (Pila.h)

Uso: Almacenar equipajes facturados (LIFO - Last In First Out).

Características:

- Puntero `tope`
- Métodos: `push()`, `pop()`, `estaVacia()`, `generarReporte()`

Complejidad temporal:

- Push: $O(1)$
- Pop: $O(1)$

```
1 #ifndef PILA_H
14 class Pila
15 {
16     Pila(/* args */)
17     {
18         cima = nullptr;
19     };
20
21     bool estaVacia()
22     {
23         return (cima == nullptr);
24     }
25
26     // Ver CIMA y contar elementos
27     T *top()
28     {
29         if (estaVacia())
30         {
31             return nullptr;
32         }
33         return cima->getDato();
34     }
35
36     int contar()
37     {
38         if (estaVacia())
39         {
40             return 0;
41         }
42
43         int contador = 0;
44         Nodo<T> *actual = cima;
45
46         while (actual != nullptr)
47         {
48             contador++;
49             actual = actual->getSiguiente();
50         }
51
52         return contador;
53     }
54
55     void push(T *dato)
56     {
57         Nodo<T> *nuevo = new Nodo<T>(dato);
58
59         if (estaVacia())
60         {
61             cima = nuevo;
62         }
63     }
64 }
```

5. Compilación y Ejecución

5.1 Compilación en Windows (MinGW)

Opción 1: Usar script de compilación

```
cd src
compile.bat
```

Opción 2: Compilación manual

```
cd src
g++ -std=c++11 -o aeropuerto.exe main.cpp Menu.cpp
```

5.2 Compilación en Linux/macOS

```
cd src
g++ -std=c++11 -o aeropuerto main.cpp Menu.cpp
```

5.3 Ejecución

Windows:

```
aeropuerto.exe
```

Linux/macOS:

```
./aeropuerto
```

6. Formato de Archivos de Entrada

6.1 aviones.json

Estructura JSON con información de cada avión:

```
{
  "vuelo": "A100",
  "numero_de_registro": "N12345",
  "modelo": "Boeing 737",
  "fabricante": "Boeing",
  "ano_fabricacion": 2015,
  "capacidad": 180,
  "peso_max_despegue": 79000,
  "aerolinea": "AirlineX",
  "estado": "Disponible"
}
```

Campos obligatorios:

- vuelo (string)
- numero_de_registro (string) - Identificador único
- estado (string) - "Disponible" o "Mantenimiento"

6.2 pasajeros.json

Estructura JSON con información de cada pasajero:

```
{
  "nombre": "John Doe",
  "nacionalidad": "Estados Unidos",
  "numero_de_pasaporte": "P12345678",
}
```

```
"vuelo": "A100",  
"asiento": 12,  
"destino": "New York",  
"origen": "Los Angeles",  
"equipaje_facturado": 2  
}
```

Campos obligatorios:

- `numero_de_pasaporte` (string) - Identificador único
- `vuelo` (string) - Para ordenamiento
- `asiento` (int) - Para ordenamiento secundario
- `equipaje_facturado` (int) - Cantidad de equipajes

6.3 Cambios.txt

Archivo de texto con comandos separados por ;

Comandos disponibles:

1. `IngresoEquipajes;`
 - Procesa el siguiente pasajero de la cola de registro
 - Lo mueve a la lista de pasajeros registrados
 - Si tiene equipaje, lo agrega a la pila
2. `MantenimientoAviones,Ingreso,{numero_registro};`
 - Mueve un avión de Disponibles a Mantenimiento
 - Ejemplo: `MantenimientoAviones,Ingreso,N12345;`
3. `MantenimientoAviones,Salida,{numero_registro};`
 - Mueve un avión de Mantenimiento a Disponibles
 - Ejemplo: `MantenimientoAviones,Salida,N12345;`

Ejemplo completo:

```
IngresoEquipajes;  
IngresoEquipajes;  
MantenimientoAviones,Ingreso,N12345;  
IngresoEquipajes;
```

7. Generación de Reportes

7.1 Proceso de Generación

El sistema genera automáticamente reportes en formato Graphviz (.dot) y los convierte a imágenes PNG.

Reportes generados:

1. `aviones_disponibles.dot/.png` - Lista circular de aviones disponibles
2. `aviones_mantenimiento.dot/.png` - Lista circular de aviones en mantenimiento
3. `cola_registro.dot/.png` - Cola de pasajeros esperando registro
4. `pasajeros_registrados.dot/.png` - Lista ordenada de pasajeros registrados
5. `pila Equipajes.dot/.png` - Pila de equipajes facturados

7.2 Ubicación de Reportes

Todos los reportes se guardan en: `reportes-generados/`

7.3 Comando Graphviz Ejecutado

Internamente, el sistema ejecuta:

```
dot -Tpng archivo.dot -o archivo.png
```

8. Uso del Sistema

8.1 Flujo Normal de Operación

1. Iniciar el programa
2. Cargar aviones (Opción 1)
 - Lee `data/aviones.json`
 - Distribuye aviones en listas según estado
3. Cargar pasajeros (Opción 2)
 - Lee `data/pasajeros.json`
 - Agrega pasajeros a la cola de registro
4. Cargar movimientos (Opción 3)
 - Lee `data/Cambios.txt`
 - Procesa comandos secuencialmente

- Registra pasajeros y mueve aviones
- 5. Consultar pasajero (Opción 4)
 - Busca por número de pasaporte
 - Muestra información completa
- 6. Visualizar reportes (Opción 5)
 - Genera todos los reportes automáticamente
 - Crea archivos .dot y .png

8.2 Ejemplo de Sesión Completa

```
=====
SISTEMA DE GESTION DE AEROPUERTO
=====

1. Cargar aviones
2. Cargar pasajeros
3. Cargar movimientos
4. Consultar pasajero
5. Visualizar reportes
0. Salir

=====

Opcion: 1

=====
CARGA DE AVIONES
=====

Cargando aviones desde ../data/aviones.json...
Se cargaron 5 aviones en total.
- Disponibles: 3
- Mantenimiento: 2
```

```

=====
COMPILANDO PROYECTO AEROPUERTO
=====

=====
COMPILACION EXITOSA
=====

Ejecutando aeropuerto.exe...

=====
FACULTAD DE INGENIERÍA
ESTRUCTURAS DE DATOS
Carné: 202303204
=====

=====
SISTEMA DE GESTION DE AEROPUERTO
=====
1. Cargar aviones
2. Cargar pasajeros
3. Cargar movimientos
4. Consultar pasajero
5. Visualizar reportes
0. Salir
=====
Opcion: 1

=====
CARGA DE AVIONES
=====
Cargando aviones desde ../data/aviones.json...
Se cargaron 5 aviones desde ../data/aviones.json
Se cargaron 5 aviones en total.
- Disponibles: 3
- Mantenimiento: 2

===== RESUMEN DE AVIONES =====
Disponibles ..... 3 avión(es)
En mantenimiento ..... 2 avión(es)
=====

Presione ENTER para continuar...

=====
SISTEMA DE GESTION DE AEROPUERTO
=====
1. Cargar aviones
2. Cargar pasajeros
3. Cargar movimientos
4. Consultar pasajero
5. Visualizar reportes
0. Salir

```

9. Consideraciones Técnicas

9.1 Gestión de Memoria

El sistema utiliza memoria dinámica para todas las estructuras:

- Operador `new` para crear objetos y nodos
- Operador `delete` para liberar memoria
- Destructores implementados en todos los controladores

Importante:

- Método `eliminar()` - Libera el objeto completo (nodo + dato)
- Método `extraer()` - Solo libera el nodo, preserva el dato para mover entre listas

9.2 Codificación UTF-8

Para soportar caracteres especiales (á, é, í, ó, ú, ñ):

```
system("chcp 65001 > nul"); // Activar UTF-8 en Windows
```

9.3 Restricciones Cumplidas

- ✓ Lenguaje: C++ con estándar C++11
- ✓ Estructuras: Implementadas manualmente (no se usan `vector`, `list`, `stack`, `queue` de STL en estructuras)
- ✓ Vectores: Solo permitidos en `JsonParser.h` para procesar JSON
- ✓ Reportes: Graphviz con formato `.dot` y conversión automática a `.png`
- ✓ Memoria dinámica: Uso correcto de `new` y `delete`

10. Solución de Problemas

10.1 Error: "dot: command not found"

Causa: Graphviz no está instalado o no está en el PATH.

Solución: Instalar Graphviz y agregar al PATH del sistema.

10.2 Error: "No se pudo abrir el archivo"

Causa: Rutas incorrectas a los archivos JSON o TXT.

Solución: Verificar que los archivos existan en `../data/` relativo a la ubicación del ejecutable.

10.3 Caracteres mal codificados

Causa: Terminal no configurada para UTF-8.

Solución: En Windows, ejecutar `chcp 65001` manualmente antes de correr el programa.

10.4 Error de compilación con C++11

Causa: Compilador no soporta `-std=c++11`.

Solución: Actualizar GCC/G++ a versión 7.0 o superior.

11. Créditos y Referencias

Desarrollado por: Christian Javier Rivas Arreaga

Carné: 202303204

Curso: Estructuras de Datos - Sección A

Universidad: Universidad de San Carlos de Guatemala

Facultad: Ingeniería

Fecha: Diciembre 2025

Tecnologías utilizadas:

- C++ (ISO/IEC 14882:2011)
- nlohmann/json (<https://github.com/nlohmann/json>)
- Graphviz (<https://graphviz.org/>)
- MinGW-w64 (Windows)
- GCC (Linux/macOS)